

Multi-Stage Docker Stack: Flask + PostgreSQL

Slim image (<150MB), non-root execution, health checks, and volume persistence

1. Requirements and how each is met

Requirement	Implementation	File
Multi-stage, image < 150MB	Builder stage creates a venv on python:3.12-alpine; runtime stage copies only the venv. Caches, tests and pip are stripped.	Dockerfile
Non-root user	Fixed UID/GID 10001, no login shell, USER app. Compose adds read_only, cap_drop ALL, no-new-privileges.	Dockerfile, docker-compose.yml
Compose: app + database	web (Flask/gunicorn) + db (postgres:16-alpine); web waits for db healthcheck.	docker-compose.yml
Health checks	web: HEALTHCHECK via Python stdlib hitting /healthz (checks DB). db: pg_isready.	Dockerfile, docker-compose.yml
Volume persistence	Named volume pgdata mounted at /var/lib/postgresql/data; test restarts the stack and verifies the counter.	scripts/test.sh

2. Project layout

```
docker-slim-app/  
|-- Dockerfile  
|-- docker-compose.yml  
|-- .dockerignore  
|-- README.md  
|-- app/  
|   |-- app.py  
|   |-- requirements.txt  
|-- scripts/  
|   |-- test.sh
```

3. How to run and verify

```
export POSTGRES_PASSWORD=choose-something  
docker compose up -d --build  
curl localhost:8000/visit  
  
# full automated verification; writes logs/build.log and logs/test-run.log  
./scripts/test.sh
```

The test script checks: no-cache build, image size under 150MB, both services healthy, non-root UID, 3 writes, *docker compose down* then *up* (volume kept), count still 3, and web turning unhealthy when the database is stopped.

4. Dockerfile

```
# syntax=docker/dockerfile:1

# ----- Stage 1: build dependencies into a virtualenv -----
FROM python:3.12-alpine AS builder
ENV PIP_NO_CACHE_DIR=1 PIP_DISABLE_PIP_VERSION_CHECK=1
WORKDIR /build
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY app/requirements.txt .
RUN pip install -r requirements.txt \
    # strip bytecode caches, tests and pip itself to shrink the venv
    && find /opt/venv -type d -name "__pycache__" -prune -exec rm -rf {} + \
    && find /opt/venv -type d -name "tests" -prune -exec rm -rf {} + \
    && pip uninstall -y pip setuptools 2>/dev/null || true

# ----- Stage 2: minimal runtime -----
FROM python:3.12-alpine AS runtime
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PATH="/opt/venv/bin:$PATH"

# Non-root user with fixed UID/GID and no login shell
RUN addgroup -g 10001 -S app && adduser -u 10001 -S -G app -H -s /sbin/nologin app

COPY --from=builder /opt/venv /opt/venv
WORKDIR /srv/app
COPY --chown=app:app app/app.py .

USER app
EXPOSE 8000

# No curl in the image: use Python's stdlib for the probe
HEALTHCHECK --interval=10s --timeout=3s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request,sys; sys.exit(0 if
        urllib.request.urlopen('http://127.0.0.1:8000/healthz', timeout=2).status==200 else 1)"

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "--access-logfile", "-", "app:app"]
```

5. docker-compose.yml

```
services:
  web:
    build:
      context: .
      target: runtime
    image: slim-web:latest
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://appuser:${POSTGRES_PASSWORD:-changeme}@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
    # hardening
    read_only: true
    tmpfs:
      - /tmp
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-changeme}
      POSTGRES_DB: appdb
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
  interval: 5s
  timeout: 3s
  retries: 10
  restart: unless-stopped

volumes:
  pgdata:
```

6. app/app.py

```
import os
import time

import psycpg
from flask import Flask, jsonify

app = Flask(__name__)
DSN = os.environ["DATABASE_URL"]

def get_conn(retries=10, delay=1.0):
    for attempt in range(retries):
        try:
            return psycpg.connect(DSN, connect_timeout=3)
        except psycpg.OperationalError:
            if attempt == retries - 1:
                raise
            time.sleep(delay)

def init_db():
    with get_conn() as conn:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS visits (id SERIAL PRIMARY KEY, ts TIMESTAMPTZ DEFAULT now())"
        )

@app.get("/healthz")
def healthz():
    try:
        with psycpg.connect(DSN, connect_timeout=2) as conn:
            conn.execute("SELECT 1")
            return jsonify(status="ok"), 200
    except Exception as exc: # unhealthy if DB unreachable
        return jsonify(status="unhealthy", error=str(exc)), 503

@app.post("/visit")
@app.get("/visit")
def visit():
    with get_conn() as conn:
        conn.execute("INSERT INTO visits DEFAULT VALUES")
        count = conn.execute("SELECT count(*) FROM visits").fetchone()[0]
    return jsonify(visits=count)

@app.get("/")
def index():
    with get_conn() as conn:
        count = conn.execute("SELECT count(*) FROM visits").fetchone()[0]
    return jsonify(message="hello from a slim container", visits=count)

init_db()
```

7. app/requirements.txt

```
flask==3.0.3
unicorn==22.0.0
psycpg[binary]==3.2.3
```

8. scripts/test.sh

```
#!/usr/bin/env bash
# Builds, verifies size/non-root/health, and proves volume persistence.
# Output is tee'd to logs/ so it can be committed as proof.
set -euo pipefail
cd "$(dirname "$0")/.."
mkdir -p logs
wait_healthy() { # $1 = service name
```

```

for _ in $(seq 1 40); do
    cid=$(docker compose ps -q "$1")
    s=$(docker inspect -f '{{.State.Health.Status}}' "$cid" 2>/dev/null || echo starting)
    [ "$s" = healthy ] && { echo "$1: healthy"; return 0; }
    sleep 2
done
echo "$1 failed to become healthy"; docker compose logs "$1"; exit 1
}

{
echo "== 1. BUILD =="
docker compose build --no-cache --progress=plain web 2>&1 | tee logs/build.log

echo; echo "== 2. IMAGE SIZE (limit 150MB) =="
bytes=$(docker image inspect slim-web:latest --format '{{.Size}}')
echo "slim-web:latest = $((bytes/1024/1024)) MB"
[ "$bytes" -lt $((150*1024*1024)) ] && echo "PASS: under 150MB" || { echo "FAIL: too large"; exit 1; }
docker history slim-web:latest

echo; echo "== 3. START STACK + HEALTHCHECKS =="
docker compose up -d
wait_healthy db
wait_healthy web

echo; echo "== 4. NON-ROOT CHECK =="
docker compose exec -T web id
[ "$(docker compose exec -T web id -u | tr -d '\r')" != "0" ] && echo "PASS: not root"

echo; echo "== 5. WRITE DATA =="
for _ in 1 2 3; do curl -fsS -X POST localhost:8000/visit; echo; done

echo; echo "== 6. RESTART CONTAINERS (down, keep volume) =="
docker compose down
docker volume ls | grep pgdata
docker compose up -d
wait_healthy db
wait_healthy web

echo; echo "== 7. VERIFY PERSISTENCE =="
out=$(curl -fsS localhost:8000/)
echo "$out"
echo "$out" | grep -q '"visits":3' && echo "PASS: data survived restart" || { echo "FAIL"; exit 1; }

echo; echo "== 8. UNHEALTHY DETECTION (stop db) =="
docker compose stop db
sleep 15
docker inspect -f 'web health: {{.State.Health.Status}}' "$(docker compose ps -q web)"
docker compose start db

echo; echo "ALL CHECKS COMPLETE"
} 2>&1 | tee logs/test-run.log

```

9. Build and test logs (evidence)

Paste the real output of `./scripts/test.sh` below (from `logs/test-run.log`), including the lines 'slim-web:latest = N MB', 'PASS: under 150MB', 'PASS: not root' and 'PASS: data survived restart'. The full build log is in `logs/build.log` in the GitHub repository.

GitHub repository: _____